

Discussion Unit: DSE Analytics

Machine Learning

- Define Machine Learning (ML)
- Overview MLlib/ML libraries
- ML process, design pattern/flow
- Upsell, customers who bought (x) bought (y)
- Predict customer churn

Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

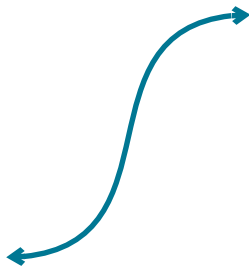
DSE Analytics; Business Intelligence (BI), versus Machine Learning (ML), Which is which

And (AI)



- © DataStax, All Rights Reserved, Confidential

- An algorithm whose accuracy increases with the amount of data it receives

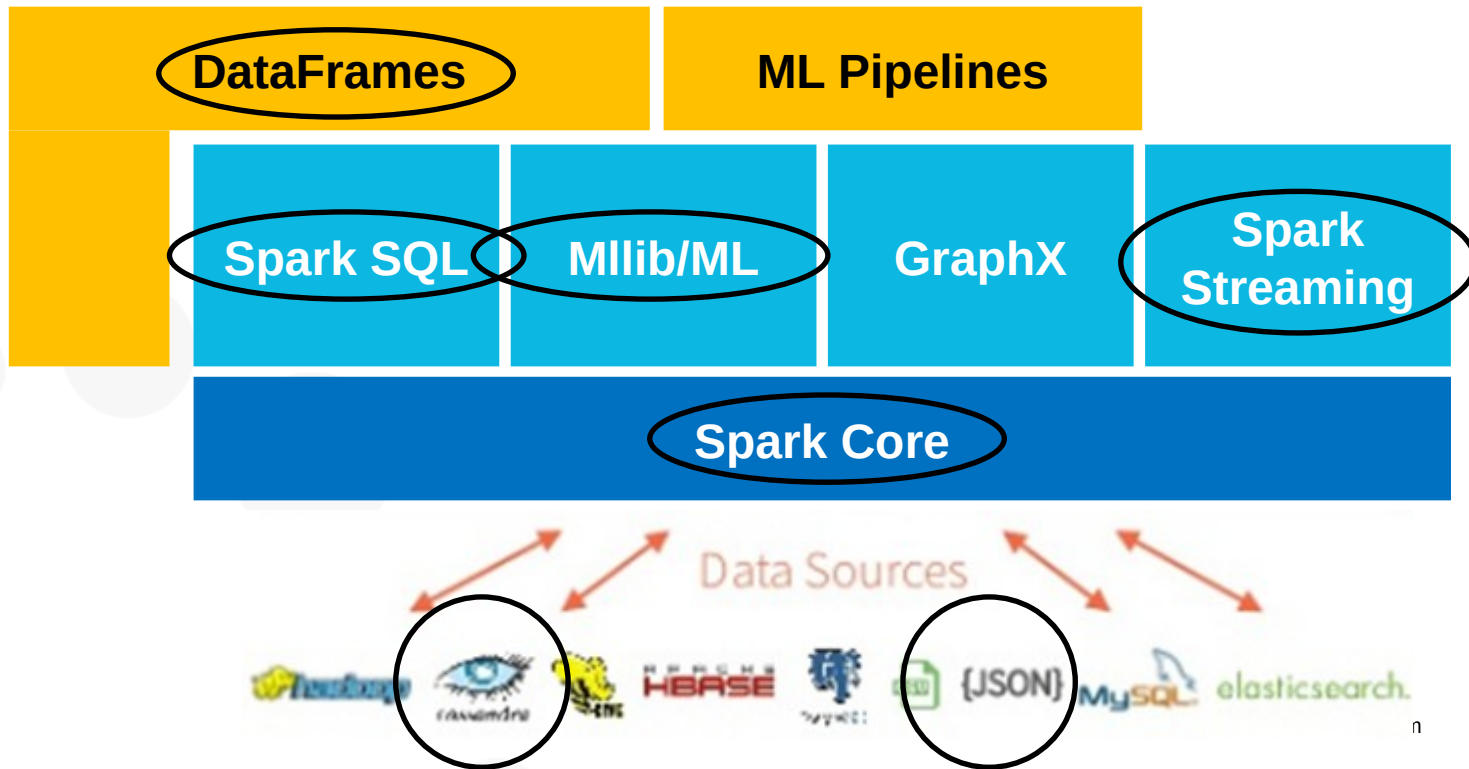


-



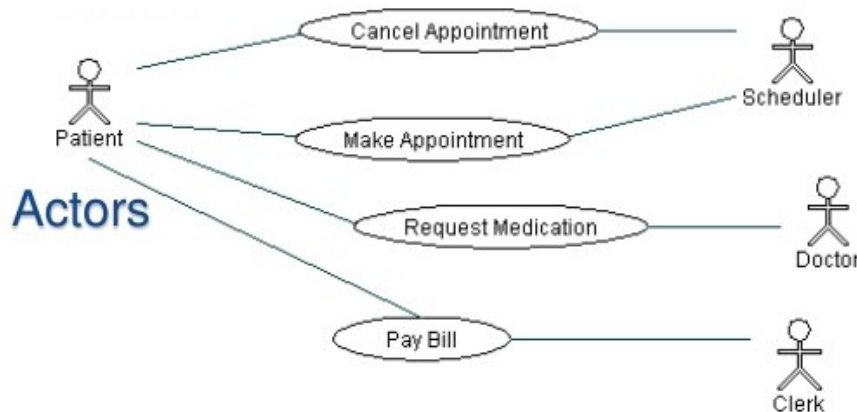
End of Discussion Lab:

DSE Analytics: 5 Major Functional Areas



DSE Analytics: ML Use Cases-

- Spam Detection/Filtering
- Anomaly/Outlier Detection
- Credit Card Fraud Detection
- Voice Recognition
- Digit/character recognition
- Internet of Things
- Gaming Analytics
- Face Detection
- Product Recommendation
- Stock Trading
- Churn



DSE Analytics: ML Use Cases, Emerging-



- Privacy Preserving Data Mining
- Author Name Disambiguation
- Recommendation Systems
- Text Mining (e.g. plagiarism)
- Sentiment Analysis
- Speech Understanding



Why ? Application ?

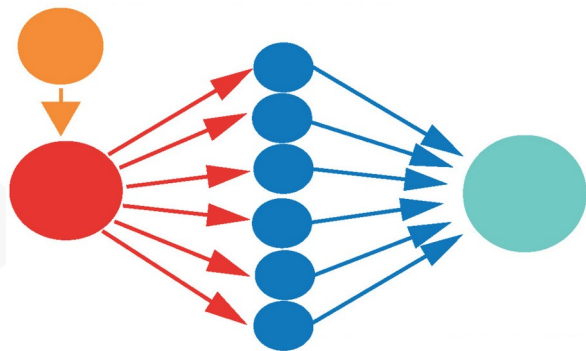
DSE Analytics: ML Enablers-

Though machine learning has been around for a while, lately there has been a surge in the need and use of machine learning

- Big data, creating a superabundance of data to analyze
- Ability for distributed systems like Spark to allow commodity computing systems to work together, instead of legacy, pricier systems
- Changes in processing power, price of RAM, allow machines to crunch through their algorithms in a reasonable amount of time

Starting with Spark 2.0, Machine Learning updated to utilize spark.ml/DataFrame libraries instead of the spark.mllib/RDD based libraries

DSE Analytics: Libraries, MLlib/RDD, ML/DataFrame



- Java, Scala, Python (Spark 0.9), R (Spark 1.5)
- Fully Parallel/Performant
- Was RDD; Spark 2.0, in maint. mode (now DF)
- Most examples still MLlib/RDD, not ML/DF
- Main doc page Url on Notes Page

DSE Analytics: Process

- Use known sample data to build a model, training phase (batch)
- Apply model to achieve business goal, scoring phase (streaming, live)
- Each ML routine can have a different model; a single array of terms, a K/V array, other, but always structured
- Data quality of the training dataset is key
- Data selection; often make assumptions on data as you build your process, be sure to note and go back to validate assumptions later
- Data munging; preprocessing, transformation



DSE Analytics: Process

ML Discovery Model Building

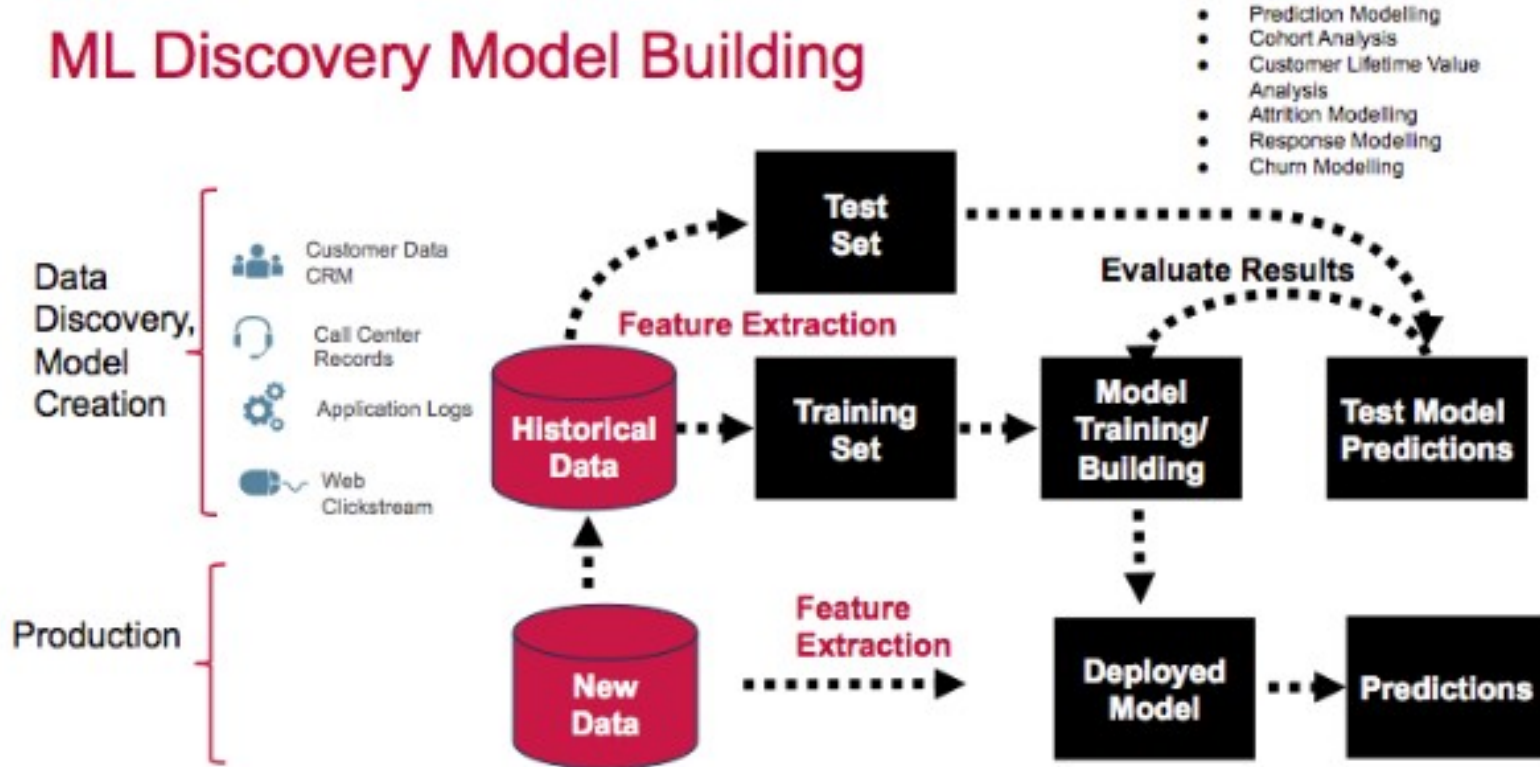


Image Source: <https://mapr.com/blog/churn-prediction-sparkml/>



+



=?



DSE Analytics: Process

- Training phase may be computationally intensive (CI), or not
- Generally, a routine with CI training phase, has a relatively computationally efficient (CE) scoring phase

[



]

?



DSE Analytics: MLLib/ML categories



Machine learning

Supervised

Classification

- Logistic regression
 - Binomial logistic regression
 - Multinomial logistic regression
- Random forest classifier
- Gradient-boosted tree classifier
- Multilayer perceptron classifier
- One-vs-Rest classifier (a.k.a. One-vs-All)

Apriori

Decision Tree Classifier

- Knn (K nearest neighbors)
- Naive Bayes

Regression

- Linear regression
- Generalized linear regression
 - Available families
- Decision tree regression
- Knn (based on data)
- Random forest regression
- Gradient-boosted tree regression
- Survival regression
- Isotonic regression

Unsupervised

Clustering

- K-means
- Decision Trees
 - Random Forests
 - Gradient boosted trees

Dimensionality Reduction

- Principle component analysis

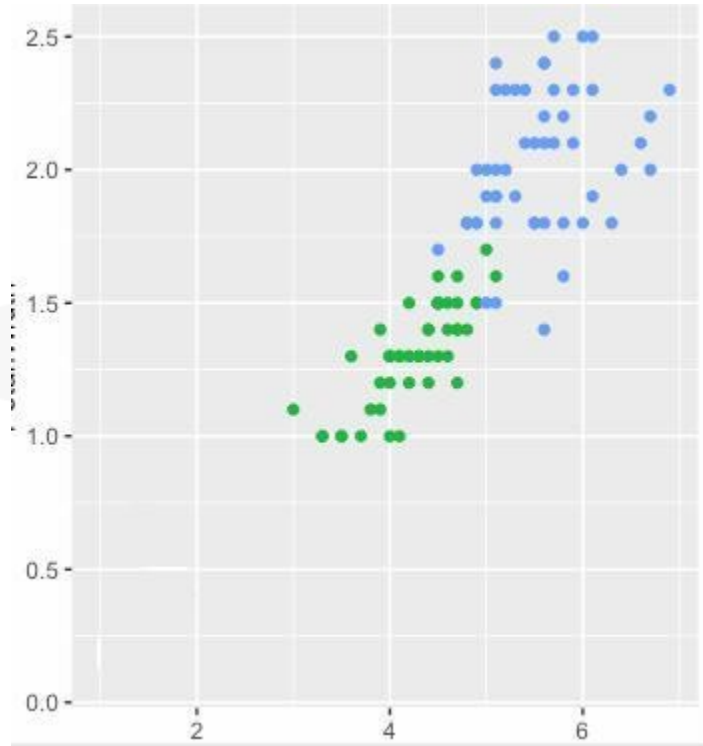
Supervised; known/labeled columns

- Case 1; Classification, Nominal values (T|F, reptile|fish|amphib, ..)
- Case 2: Regression, Infinite numeric values
- Example case 1; frequency pattern matching
- Knn; (seminal use case/example, bird identification)

Unsupervised; unlabeled columns

- Generate clusters of data
- Often used for discovery, to gain discovery, find outliers

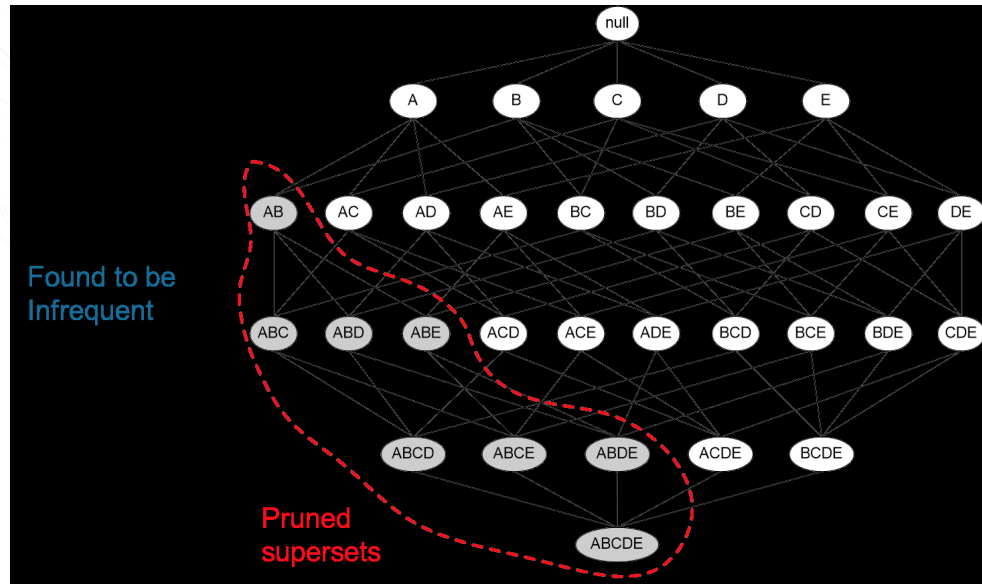
Sample: Supervised, Classification/Regression, Knn



- Attribution, graphing
- x, y, (z ?), .. **input, "features"**
- Real world; normally 70-90% overlapping in first attempts
- Outliers, patterns, more
- Next step: add/remove columns
- Ask for more/fewer groups, **"labels"**
- Example; find relationship between columns involved in churn- What should you be looking for out of 100+ enterprise database columns

Machine Learning:

Upselling / Customers who bought (x) also bought (y),
using the Apriori Algorithm



Machine Learning: Apriori Algorithm



Order Set

Tx num	Items sold
0	soy milk, lettuce
1	lettuce, diapers , wine, chard
2	soy milk, diapers , wine, OJ
3	lettuce, soy milk, diapers , wine
4	lettuce, soy milk, diapers , OJ

Item Set



Item Set

Apriori

- **Support:** percentage of **dataset** that contains given **item set**.
Soy-milk/diapers is 3 (item set) / 5 (dataset).
- **Confidence:** support (A/B) / support (A)
(diapers, soy-milk) / (diapers) == (3/5) / (4/5) == 75%

Low support ? We just don't have frequent data,
not a good candidate for this type of analysis ?
Confidence, with good support, useful



Some similarity across, but
each ML routine may have
its own (vocabulary)

- A dataset with N items can generate $2^N - 1$ (power of) possible item sets
 $2^{100} - 1 == 1267650600228229401496703205375$



Items

- We need to prune/optimize

Apriori

- How-

- Generate a list of all unique items (singles)
- Generate a list of all unique (item pairs, and higher)

soy -> lettuce (tx 0)

soy -> diapers (tx 2)

soy -> diapers, wine (tx 2)

soy -> diapers, wine, OJ (tx 2)

...

Calculate *support* and *confidence* for each

- *Apriori*-

- If A/B is (in)frequent, then A/B/C, A/B/D, and A/B/C/D are also (in)frequent (so discard them)

- **Association rules: antecedent -> consequent**

diapers -> chard $(1/5 \text{ diapers-chard}) / (4/5 \text{ diapers}) = 25\%$

chard -> diapers $(1/5 \text{ diapers-chard}) / (1/5 \text{ chard}) = 100\%$

Is it statistically significant

Created from 2 or more items

Working Goal:

From: Maury_Atwater

To: DSE_HOTSHOT

Subject: Need this now !!!

We need to start upselling ! CSV file; 10k orders, 43k items sold, 170 unique items, 4.4 items per transaction.

We need a set of association rules to apply on the Web site !

-MA



Maury Atwater, President
of Atwater's

A new dataset, also grocery:

- 9,835 Customer orders
 - 43,367 Items sold
 - 169 Unique items
 - 4.4 Average items per order
-
- With > 1% Support, 523 Association Rules created

Apriori: New data set

Selected (culled data):

Old/Small set:

chard	--> wine	<u>conf: 1.0</u>
chard	--> lettuce, wine	<u>conf: 1.0</u>
chard	--> diapers, wine	<u>conf: 1.0</u>
OJ, wine	--> soy milk, diapers	<u>conf: 1.0</u>

Large set:

yogurt	--> whole milk	<u>conf: 0.401603498542</u>
whole milk	--> yogurt	<u>conf: 0.219259848786</u>
other vegetables	--> whole milk	<u>conf: 0.38675775092</u>
whole milk	--> other vegetables	<u>conf: 0.292877039395</u>
coffee	--> other vegetables	<u>conf: 0.231173380035</u>
coffee	--> rolls/buns	<u>conf: 0.189141856392</u>
coffee	--> whole milk	<u>conf: 0.322241681261</u>

Smallest/largest confidence, > 1% confidence, > 1% support

whole milk	--> hard cheese	<u>conf: 0.0393951452447</u>
butter	--> whole milk	<u>conf: 0.497247706422</u>

Apriori: If you hand coded in Python, other

- All manual Python code, 809 lines
- Large data set, killed it after 1 hour
- Using Spark/Python libraries, 54 lines of code
- Large data set, laptop/VM, 2 cores, 20 seconds



Coding the Apriori

Customers who bought (x)
bought (y)

Apriori: In Python, Pg 1 of 2

```
from pyspark.mllib.fpm import FPGrowth  
from pyspark import SparkContext
```

```
sc = SparkContext()
```

```
l_datafile = sc.textFile("10_grocery.csv")  
l_orders = l_datafile.map(lambda line: line.strip().split(','))
```



What is the 0.01 here ?

```
l_model = FPGrowth.train(l_orders, minSupport=0.01, numPartitions=1)
```

```
l_results = l_model.freqItemsets().collect()  
# l_results is of type: <class 'collections.FreqItemset'>
```

Apriori: In Python, Pg 2 of 2

```
for l_result in l_results:  
    print str(l_result[0]) + " " + str(float(l_result[1]) / 100)
```

```
# From Spark,  
# [u'whole milk'] 0.2513  
# [u'other vegetables'] 0.1903  
# [u'rolls/buns'] 0.1809  
#  
# From pure Python,  
# frozenset(['whole milk']) 0.255516014235  
# frozenset(['other vegetables']) 0.193492628368  
# frozenset(['rolls/buns']) 0.183934926284
```



0 == (label)
1 == (support)

Python currently: You have
to calculate confidence and
association rules yourself.
Not awful, but ..

Apriori: In Scala, 1/6

```
package com.datastax.enablement.bootcamp

import org.apache.spark.{SparkConf, SparkContext}

import org.apache.spark.mllib.fpm.FPGrowth
import org.apache.spark.rdd.RDD

import org.apache.spark.mllib.fpm.AssociationRules
```



And others if you
expand scope

Apriori: In Scala, 2/6

```
object App {
```

```
  def main(args: Array[String]) {
```

```
    val sc = new SparkContext(new SparkConf().  
      setAppName("My App"))
```

```
    val data = sc.textFile("file:///opt/dse_ml/10_grocery.csv")
```

```
    val transactions: RDD[Array[String]] = data.  
      map(s => s.trim.split(', '))
```

```
    // Sample data lines from CSV
```

```
    //   citrus fruit,semi-finished bread,margarine,ready soups
```

```
    //   tropical fruit,yogurt,coffee
```



Any ML yet ?

Apriori: In Scala, 3/6

```
val fpg = new FPGrowth().setMinSupport(0.01).  
    setNumPartitions(1)  
  
val model = fpg.run(transactions)  
  
model.freqItemsets.collect().foreach { itemset =>  
    println(itemset.items.mkString("[", ",", "]") +  
        ", " + itemset.freq)  
}
```



Python was
train() ?

```
// {canned vegetables}: 106  
// {pork}: 567  
//
```



Data from println()

```
// 99 {curd,yogurt,whole milk}  
// 2513 {whole milk}
```



The actual high/low

```
// class org.apache.spark.mllib.fpm.FPGrowth$FreqItemset;
```

Apriori: In Scala, 4/6

```
val minConfidence = 0.8

model.generateAssociationRules(minConfidence).
  collect().foreach { rule =>
    println(
      rule.antecedent.mkString("[", ",", "]")
      + " => " + rule.consequent.mkString("[", ",", "]")
      + ", " + rule.confidence
    )
  }
```



Difference between this
page and the next page ?

Apriori: In Scala, 5/6

```
println("")  
println("")
```

```
val ar = new AssociationRules().setMinConfidence(0.1)  
val results = ar.run(model.freqItemsets)
```

```
results.collect().foreach { rule =>  
  println "[" + rule.antecedent.mkString(",")  
    + "=>"  
    + rule.consequent.mkString(",") + "], " +  
    rule.confidence  
  )  
}
```

Apriori: In Scala, 6/6

```
    sc.stop()
    sys.exit(0)
  }
}

// High low,
// 0.10074626865671642 {root vegetables} => {pastry}
// 0.10167910447761194 {root vegetables} => {margarine}
// ...
// 0.5845410628019324 {tropical fruit,root vegetables} => {other vegetables}
// 0.5862068965517241 {citrus fruit,root vegetables} => {other vegetables}
//
// class org.apache.spark.mllib.fpm.AssociationRules$Rule
```



Customer's who bought (x) bought (y) - DataStax Enterprise

Persons who bought (x)..

Notes:

This TAB demonstrates the proposed solution: Persons who bought (x) also bought (y).

We use this capability to encourage a higher revenue per transaction.

The drop down list box is pre-populated with each unique single item this store sells. On change, a query returns all of the matching rules which contain the upse

The diapers product offers the most interesting data.

Select a product:

Wine

Apriori: Application

To Buy	To Sell	Confidence
wine	diapers	1
wine	lettuce	0.6666666865348816
wine	lettuce diapers	0.6666666865348816
wine	soy-milk	0.6666666865348816
wine	soy-milk diapers	0.6666666865348816

How would you know this

- Url on Notes page
- And this example

spark.ml package

- Overview: estimators, transformers and pipelines
- Extracting, transforming and selecting features
- Classification and Regression
- Clustering
- Advanced topics

spark.mllib package

- Data types
- Basic statistics
- Classification and regression
- Collaborative filtering
- Clustering
- Dimensionality reduction
- Feature extraction and transformation
- Frequent pattern mining
 - FP-growth
 - association rules
 - PrefixSpan
- Evaluation metrics
- PMML model export
- Optimization (developer)

Frequent Pattern Mining - spark.mllib

Mining frequent items, itemsets, subsequences, or other substructures is usually among the first steps to analyze a large-scale dataset, which has been an active research topic in data mining for years. We refer users to Wikipedia's [association rule learning](#) for more information. `spark.mllib` provides a parallel implementation of FP-growth, a popular algorithm to mining frequent itemsets.

FP-growth

The FP-growth algorithm is described in the paper [Han et al., Mining frequent patterns without candidate generation](#), where "FP" stands for frequent pattern. Given a dataset of transactions, the first step of FP-growth is to calculate item frequencies and identify frequent items. Different from [Apriori-like](#) algorithms designed for the same purpose, the second step of FP-growth uses a suffix tree (FP-tree) structure to encode transactions without generating candidate sets explicitly, which are usually expensive to generate. After the second step, the frequent itemsets can be extracted from the FP-tree. In `spark.mllib`, we implemented a parallel version of FP-growth called PFP, as described in [Li et al., PFP: Parallel FP-growth for query recommendation](#). PFP distributes the work of growing FP-trees based on the suffixes of transactions, and hence more scalable than a single-machine implementation. We refer users to the papers for more details.

`spark.mllib`'s FP-growth implementation takes the following (hyper-)parameters:

- `minSupport`: the minimum support for an itemset to be identified as frequent. For example, if an item appears 3 out of 5 transactions, it has a support of $3/5=0.6$.
- `numPartitions`: the number of partitions used to distribute the work.

Examples

Scala Java Python

`FPGrowth` implements the FP-growth algorithm. It take a RDD of transactions, where each transaction is an Array of items of a generic type. Calling `FPGrowth`, run with transactions returns an `FPGrowthModel` that stores the frequent itemsets with their frequencies. The following example illustrates how to mine frequent itemsets and association rules (see [Association Rules](#) for details) from transactions.

Refer to the [FPGrowth Scala docs](#) for details on the API.

```
import org.apache.spark.mllib.fpm.FPGrowth
import org.apache.spark.rdd.RDD

val data = sc.textFile("data/mllib/sample_fpgrowth.txt")

val transactions: RDD[Array[String]] = data.map(s => s.trim.split(' '))

val fpg = new FPGrowth()
```

Association Rules

Scala Java

[AssociationRules](#) implements a parallel rule generation algorithm for constructing rules that have a single item as the consequent.

Refer to the [AssociationRules Scala docs](#) for details on the API.

```
import org.apache.spark.mllib.fpm.AssociationRules
import org.apache.spark.mllib.fpm.FPGrowth.FreqItemset

val freqItemsets = sc.parallelize(Seq(
  new FreqItemset(Array("a"), 15L),
  new FreqItemset(Array("b"), 35L),
  new FreqItemset(Array("a", "b"), 12L)
))

val ar = new AssociationRules()
  .setMinConfidence(0.8)
val results = ar.run(freqItemsets)

results.collect().foreach { rule =>
  println("[ " + rule.antecedent.mkString(",")
    + "=>"
    + rule.consequent.mkString(",") + "], " + rule.confidence)
}
```

Find full example code at "examples/src/main/scala/org/apache/spark/examples/mllib/AssociationRulesExample.scala" in the Spark repo.

PrefixSpan

How would you know this



No Python TAB ?

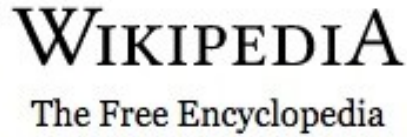
Apriori: Summary

- order set, item set
- support
- confidence
- association rules
- antecedent, consequent
- C1, candidate item set of size 1
- C2, ..
- L2, frequent item set (scored, size 2)
- FPGrowth.run()
 - FPGrowth.train()
 - freqItemsets
 - (0 == label, 1 == confidence)
 - freqItemsets.items
 - freqItemsets.freq
 - AssociationRules.run()
 - antecedent
 - consequent
 - confidence

Machine Learning:

Predicting Customer Churn
using Decision Tree
Classifier





Reference Url:https://en.wikipedia.org/wiki/Customer_attrition

Customer Churn/Attrition

Banks, telephone service companies, Internet service providers, pay TV companies, insurance firms, and alarm monitoring services, often use customer attrition analysis and **customer attrition rates** as one of their **key business metrics** (along with cash flow, EBITDA, etc.) because the **cost of retaining** an existing customer is far less than acquiring a new one.

Companies from these sectors often have customer service branches which attempt to win back defecting clients, ...

Churn: Data required

In order to understand the customer, a number of factors can be analyzed, such as:

- Customer demographic data (age, marital status, etc.)
- Sentiment analysis of social media
- Customer usage patterns and geographical usage trends
- Calling-circle data
- Browsing behavior from clickstream logs
- Support call center statistics
- Historical data that show patterns of behavior that suggest churn

Churn Example: USA, Dish Network



- Customer/360 (Need the data)
- You rebooted your modem twice that morning
- You accessed the pricing page on the Web site
- You queried "cancel service" on the Web page
- Now you're in the VRU/800-number
- Which of 30,000 CSRs do we route you to ?

DataStax
Internal Use
Only

Churn: Atwater's data

CREATE TABLE customer

customer_num

fname

lname

company

address1

address2

city

state

zipcode

phone



CREATE TABLE orders

order_num

order_date

customer_num

ship_instruct

backlog

po_num

ship_date

ship_weight

ship_charge

paid_date

CREATE TABLE items

item_num

order_num

stock_num

manu_code

quantity

total_price

Churn

SIGKDD

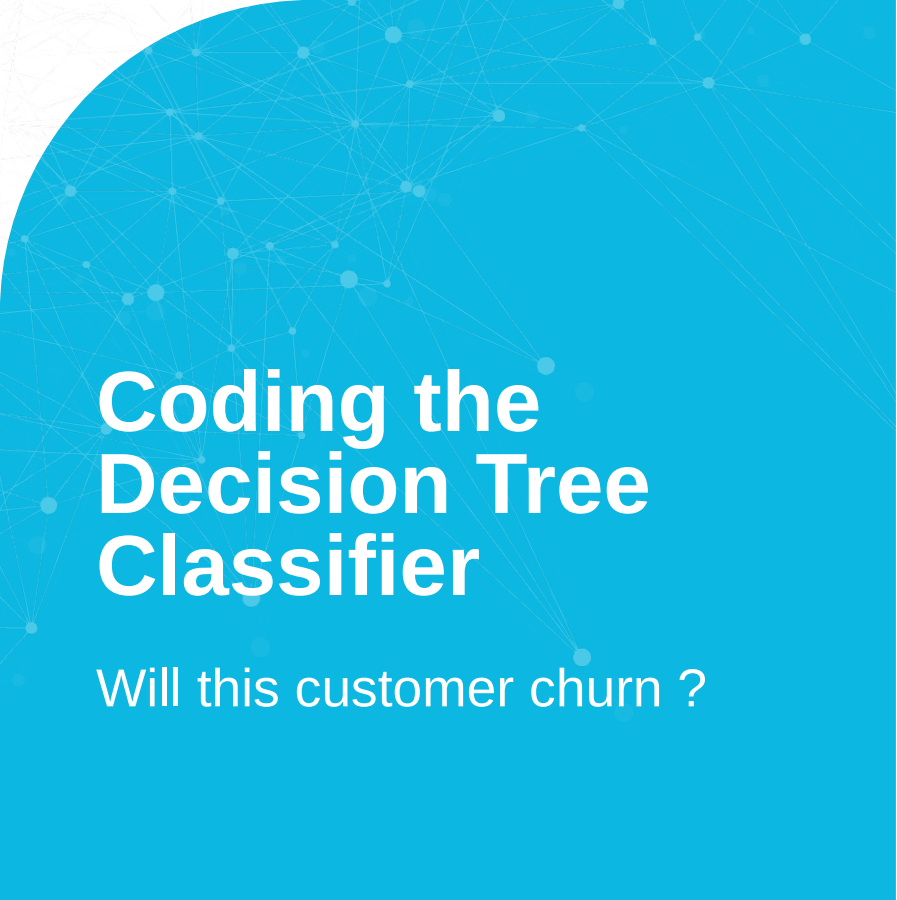
Sig.K•D•D \ 'sig-kā-dē-dē\ **Noun** (20 c) The Association for Computing Machinery's Special Interest Group on Knowledge Discovery and Data Mining.

- Yearly contest, specifically 2009, focus was Churn
- "The challenge (was) to beat the in-house system developed by Orange Labs." A.k.a, Orange Telecomm (<https://www.orange.com/en/home>)
- 3000+ real data rows (customer ID cleansed)
- 262 lines of Scala
- Will this customer churn ?

Churn: Data model, telecomm

State	Total eve calls
Account length	Total eve charge
Area code	Total night minutes
International plan	Total night calls
Voice mail plan	Total night charge
Number vmail messages	Total intl minutes
Total day minutes	Total intl calls
Total day calls	Total intl charge
Total day charge	Customer service calls
Total eve minutes	Churn (Did they churn (label): T F)

LA,117,408,No,No,0,184.5,97,31.37,351.6,80,29.89,215.8,90,9.71,8.7,4,2.35,1,False
IN,65,415,No,No,0,129.1,137,21.95,228.5,83,19.42,208.8,111,9.4,12.7,6,3.43,4,True



Coding the Decision Tree Classifier

Will this customer churn ?

Churn: In Scala 1 of 14

```
package com.datastax.enablement.bootcamp
```

```
import org.apache.spark.sql.Session
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._
import org.apache.spark.sql.Dataset
import org.apache.spark.ml.Pipeline
import org.apache.spark.ml.classification.DecisionTreeClassifier
import org.apache.spark.ml.classification.DecisionTreeClassificationModel
import org.apache.spark.mllib.evaluation.BinaryClassificationMetrics
import org.apache.spark.ml.evaluation.BinaryClassificationEvaluator
import org.apache.spark.ml.feature.StringIndexer
import org.apache.spark.ml.tuning.ParamGridBuilder
import org.apache.spark.ml.tuning.CrossValidator
import org.apache.spark.ml.feature.VectorAssembler
```

Churn: In Scala 2 of 14

```
object App {
```

```
  case class Account
```

```
  (
```

```
    state: String,
```

```
    len: Integer,
```

```
    acode: String,
```

```
    intlplan: String,
```

```
    vplan: String,
```

```
    numvmail: Double,
```

```
    tdmns: Double,
```

```
    tdcalls: Double,
```

```
    tdcharge: Double,
```

```
    temns: Double,
```

```
    tecalls: Double,
```

```
    techarge: Double,
```

```
    tnmins: Double,
```

```
    tncalls: Double,
```

```
    tncharge: Double,
```

```
    timns: Double,
```

```
    ticalls: Double,
```

```
    ticharge: Double,
```

```
    numcs: Double,
```

```
    churn: String
```

```
  )
```

Churn: In Scala 3 of 14

```
val schema = StructType(Array(  
  StructField("state", StringType, true),  
  StructField("len", IntegerType, true),  
  StructField("acode", StringType, true),  
  StructField("intlplan", StringType, true),  
  StructField("vplan", StringType, true),  
  StructField("numvmail", DoubleType, true),  
  StructField("tdmins", DoubleType, true),  
  ...  
  StructField("tncharge", DoubleType, true),  
  StructField("timins", DoubleType, true),  
  StructField("ticalls", DoubleType, true),  
  StructField("ticharge", DoubleType, true),  
  StructField("numcs", DoubleType, true),  
  StructField("churn", StringType, true)  
))
```

Churn: In Scala 4 of 14

```
def main(args: Array[String]) {  
  
    val spark: SparkSession = SparkSession.builder().  
        appName("churn").getOrCreate()  
  
    import spark.implicits._  
  
    val train: Dataset[Account] = spark.read.option(  
        "inferSchema", "false").  
        schema(schema).csv(  
            "file:///opt/dse_ml/churn-bigml-80.csv").  
        as[Account]  
    train.take(1)  
    train.cache  
    //  
    println(train.count)  
}
```

Churn: In Scala 5 of 14

```
val test: Dataset[Account] = spark.read.option(
  "inferSchema", "false").
  schema(schema).csv(
    "file:///opt/dse_ml/churn-bigml-20.csv").
  as[Account]
test.take(2)
println(test.count)
//
test.cache

train.printSchema()
train.show

train.createOrReplaceTempView("account")
spark.catalog.cacheTable("account")

train.groupBy("churn").count.show
```

Churn: In Scala 6 of 14

```
val fractions = Map("False" -> .17, "True" -> 1.0)

// Here we're keeping all instances of the Churn=True class,
// but downsampling the Churn=False class to a fraction of 388/2278.
val strain = train.stat.sampleBy("churn", fractions, 36L)
//
strain.groupBy("churn").count.show
val ntrain = strain.drop("state").drop("acode").
    drop("vplan").drop("tdcharge").drop("techarge")
println(ntrain.count)
ntrain.show
```

Description: Returns a stratified sample without replacement based on the fraction given on each stratum.

Usage: `sampleBy(x, col, fractions, seed)`

Spark: `sampleBy()`

// S4 method for signature 'SparkDataFrame,character,list,numeric'

`sampleBy(x, col, fractions, seed)`

Arguments:

x	A SparkDataFrame
col	column that defines strata
fractions	A named list giving sampling fraction for each stratum. If a stratum is not specified, we treat its fraction as zero.
seed	random seed

Value: A new SparkDataFrame that represents the stratified sample

Churn: In Scala 7 of 14

```
val ipindexer = new StringIndexer().  
  setInputCol("intlplan").  
  setOutputCol("iplanIndex")  
val labelindexer = new StringIndexer().  
  setInputCol("churn").  
  setOutputCol("label")  
val featureCols = Array("len", "iplanIndex",  
  "numvmail", "tdmins", "tdcalls", "temins",  
  "tecalls", "tnmins", "tncalls", "timins",  
  "ticalls", "numcs")  
  
val assembler = new VectorAssembler().  
  setInputCols(featureCols).  
  setOutputCol("features")  
  
val dTree = new DecisionTreeClassifier().  
  setLabelCol("label").  
  setFeaturesCol("features")
```

Spark: StringIndexer()

"The ML package needs data to be put in a (label: Double, features: Vector) DataFrame format with correspondingly named fields. We set up a pipeline to pass the data through 3 transformers in order to extract the features: 2 StringIndexers and a VectorAssembler. We use the StringIndexers to convert the String Categorical feature intlplan and label into number indices. Indexing categorical features allows decision trees to treat categorical features appropriately, improving performance.



tdcalls	tdcharge	temins	teccalls	techarge	trmins	trccalls	trcharge	timins	ticcalls	ticharge	numos	churn	iplanIndex	label	features
97	31.37	351.6	80	29.89	215.8	90	9.71	8.7	4	2.35	1	False	0	1	> [1,12,], [117,0,0,184.5,97,351.6,80,215.8,90,8.7,4,1]]
137	21.95	228.5	83	19.42	208.8	111	9.4	12.7	6	3.43	4	True	0	0	> [1,12,], [65,0,0,129.1,137,228.5,83,208.8,111,12.7,6,
67	56.59	317.8	97	27.01	160.6	128	7.23	5.4	9	1.46	4	True	0	0	> [1,12,], [161,0,0,332.9,67,317.8,97,160.6,128,5.4,9,4
103	18.77	137.3	102	11.67	189.6	105	8.53	7.7	6	2.08	2	False	0	1	> [1,12,], [111,0,0,110.4,103,137.3,102,189.6,105,7.7,6
117	20.28	215.1	109	18.28	178.7	90	8.04	11.1	1	3	1	False	0	1	> [1,12,], [49,0,0,119.3,117,215.1,109,178.7,90,11.1,1,

Spark: VectorAssembler()

VectorAssembler is a transformer that combines a given list of columns into a single vector column. It is useful for combining raw features and features generated by different feature transformers into a single feature vector, in order to train ML models like logistic regression and decision trees. VectorAssembler accepts the following input column types: all numeric types, boolean type, and vector type. In each row, the values of the input columns will be concatenated into a vector in the specified order.

Assume that we have a DataFrame with the columns `id`, `hour`, `mobile`, `userFeatures`, and `clicked`:

id	hour	mobile	userFeatures	clicked
0	18	1.0	[0.0, 10.0, 0.5]	1.0

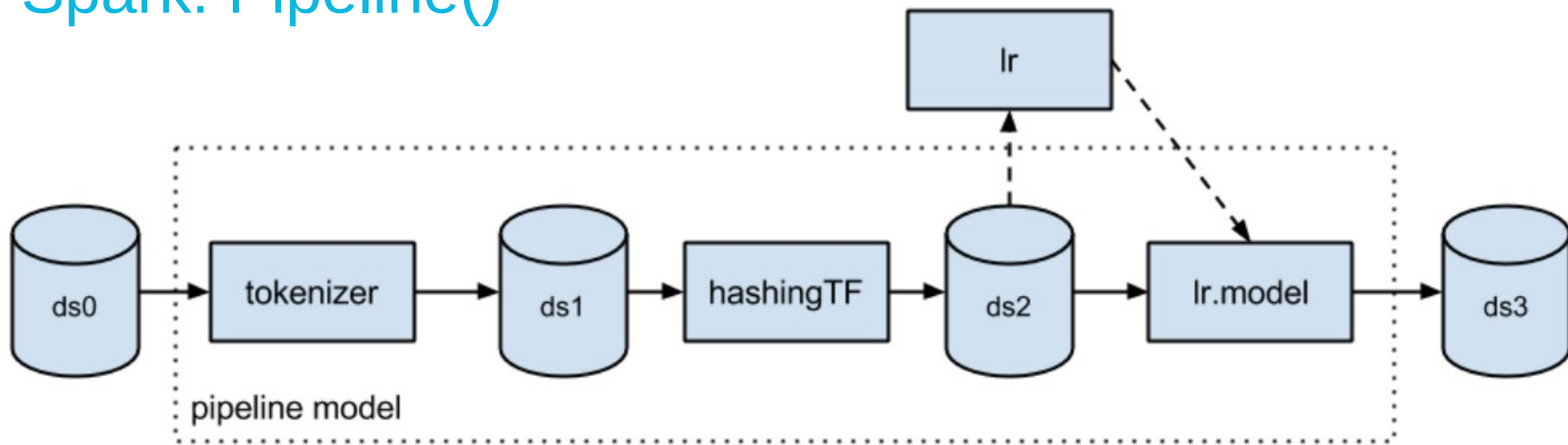
`userFeatures` is a vector column that contains three user features. We want to combine `hour`, `mobile`, and `userFeatures` into a single feature vector called `features` and use it to predict `clicked` or not. If we set VectorAssembler's input columns to `hour`, `mobile`, and `userFeatures` and output column to `features`, after transformation we should get the following DataFrame:

id	hour	mobile	userFeatures	clicked	features
0	18	1.0	[0.0, 10.0, 0.5]	1.0	[18.0, 1.0, 0.0, 10.0, 0.5]

Churn: In Scala 8 of 14

```
// Chain indexers and tree in a Pipeline.  
val pipeline = new Pipeline().  
    setStages(Array(ipindexer, labelindexer,  
        assembler, dTree))  
  
// Search through decision tree's maxDepth  
// parameter for best model  
val paramGrid = new ParamGridBuilder().  
    addGrid(dTree.maxDepth, Array(2, 3, 4,  
        5, 6, 7)).build()  
  
val evaluator = new BinaryClassificationEvaluator().  
    setLabelCol("label").  
    setRawPredictionCol("prediction")
```

Spark: Pipeline()



The ML Pipelines is a High-Level API for MLlib that lives under the “spark.ml” package. A pipeline consists of a sequence of stages. There are two basic types of pipeline stages: Transformer and Estimator. A Transformer takes a dataset as input and produces an augmented dataset as output. E.g., a tokenizer is a Transformer that transforms a dataset with text into an dataset with tokenized words. An Estimator must be first fit on the input dataset to produce a model, which is a Transformer that transforms the input dataset. E.g., logistic regression is an Estimator that trains on a dataset with labels and features and produces a logistic regression model.

Spark: (Many)

ML Tuning: model selection and hyperparameter tuning

This section describes how to use MLlib's tooling for tuning ML algorithms and Pipelines. Built-in Cross-Validation and other tooling allow users to optimize hyperparameters in algorithms and Pipelines.

Table of contents

- [Model selection \(a.k.a. hyperparameter tuning\)](#)
- [Cross-Validation](#)
- [Train-Validation Split](#)

Model selection (a.k.a. hyperparameter tuning)

An important task in ML is *model selection*, or using data to find the best model or parameters for a given task. This is also called *tuning*. Tuning may be done for individual Estimators such as `LogisticRegression`, or for entire Pipelines which include multiple algorithms, featurization, and other steps. Users can tune an entire Pipeline at once, rather than tuning each element in the Pipeline separately.

MLlib supports model selection using tools such as [CrossValidator](#) and [TrainValidationSplit](#). These tools require the following items:

- **Estimator**: algorithm or Pipeline to tune
- Set of ParamMaps: parameters to choose from, sometimes called a “parameter grid” to search over
- **Evaluator**: metric to measure how well a fitted Model does on held-out test data

- `ParamGridBuilder()`
- `BinaryClassificationBuilder()`
- `CrossValidator()`

Churn: In Scala 9 of 14

```
// Set up 3-fold cross validation
val crossval = new CrossValidator().
  setEstimator(pipeline).
  setEvaluator(evaluator).
  setEstimatorParamMaps(paramGrid).
  setNumFolds(3)

val cvModel = crossval.fit(ntrain)

val bestModel = cvModel.bestModel
println("The Best Model and Parameters:\n-----")
println(bestModel.asInstanceOf[
  org.apache.spark.ml.PipelineModel].stages(3))
//
bestModel.asInstanceOf[org.apache.spark.ml.PipelineModel].
  stages(3).
  extractParamMap
```

Churn: In Scala 10 of 14

```
val treeModel = bestModel.asInstanceOf[
    org.apache.spark.ml.PipelineModel].
    stages(3).asInstanceOf[
        DecisionTreeClassificationModel]
println("Learned classification tree model:\n" +
    treeModel.toDebugString)

val predictions = cvModel.transform(test)
val accuracy = evaluator.evaluate(predictions)
evaluator.explainParams()
```

Churn: In Scala 11 of 14

```
val predictionAndLabels = predictions.select(
  "prediction", "label").
  rdd.map(x => (x(0).asInstanceOf[Double],
    x(1).asInstanceOf[Double]))
val metrics = new BinaryClassificationMetrics(predictionAndLabels)
println("area under the precision-recall curve: " +
metrics.areaUnderPR)
println("area under the receiver operating characteristic (ROC) curve :
" +
  metrics.areaUnderROC)

println(metrics.fMeasureByThreshold())

val result = predictions.select("label", "prediction",
  "probability")
result.show
```

Churn: In Scala 12 of 14

```
val lp = predictions.select("label", "prediction")
val counttotal = predictions.count()
val correct = lp.filter($"label" === $"prediction").count()
val wrong = lp.filter(not($"label" === $"prediction")).count()
val ratioWrong = wrong.toDouble / counttotal.toDouble
val ratioCorrect = correct.toDouble / counttotal.toDouble
val truep = lp.filter($"prediction" === 0.0).filter(
  $"label" === $"prediction").count() / counttotal.toDouble
val truen = lp.filter($"prediction" === 1.0).filter(
  $"label" === $"prediction").count() / counttotal.toDouble
val falsep = lp.filter($"prediction" === 1.0).filter(not(
  $"label" === $"prediction")).count() / counttotal.toDouble
val falsen = lp.filter($"prediction" === 0.0).filter(not(
  $"label" === $"prediction")).count() / counttotal.toDouble
```

Churn: In Scala 13 of 14

```
println("counttotal : " + counttotal)
println("correct : " + correct)
println("wrong: " + wrong)
println("ratio wrong: " + ratioWrong)
println("ratio correct: " + ratioCorrect)
println("ratio true positive : " + truep)
println("ratio false positive : " + falsep)
println("ratio true negative : " + truen)
println("ratio false negative : " + falsen)

println("wrong: " + wrong)
```

Churn: In Scala 14 of 14

```
val equalp = predictions.selectExpr(  
    "double(round(prediction)) as prediction", "label",  
    """CASE double(round(prediction)) = label WHEN true  
        then 1 ELSE 0 END as equal"""  
)  
  
equalp.show  
  
spark.stop()  
sys.exit(0)  
  
}  
  
}
```

Churn, Decision Tree: Select Output-

The Best Model and Parameters:

DecisionTreeClassificationModel (uid=dtc_d2da400284df) of depth 5 with 53 nodes

Learned classification tree model:

DecisionTreeClassificationModel (uid=dtc_d2da400284df) of depth 5 with 53 nodes

```
If (feature 11 <= 3.0)
  If (feature 3 <= 222.4)
    If (feature 1 in {1.0})
      If (feature 9 <= 13.1)
        If (feature 10 <= 2.0)
          Predict: 0.0
        Else (feature 10 > 2.0)
          Predict: 1.0
      Else (feature 9 > 13.1)
        Predict: 0.0
    Else (feature 1 not in {1.0})
```

Churn, Decision Tree: Select Output-

```
counttotal : 667  
correct : 574  
wrong: 93  
ratio wrong: 0.13943028485757122  
ratio correct: 0.8605697151424287  
ratio true positive : 0.1184407796101949  
ratio false positive : 0.0239880059970015  
ratio true negative : 0.7421289355322339  
ratio false negative : 0.11544227886056972
```

Churn, Decision Tree: Select Output-

area under the precision-recall curve: 0.9747578698231796

area under the receiver operating characteristic (ROC) curve :

0.8484817813765183

MapPartitionsRDD[1233] at map at BinaryClassificationMetrics.scala:214

```
+-----+-----+-----+
|label|prediction|probability|
+-----+-----+-----+
| 1.0|1.0|[0.10676156583629...|
| 0.0|0.0|[0.91666666666666...|
| 0.0|0.0|[1.0,0.0]|
| 1.0|1.0|[0.10676156583629...|
| 1.0|1.0|[0.10676156583629...|
| 1.0|1.0|[0.1,0.9]|
| 1.0|1.0|[0.27027027027027...|
...

```

How would you know this

- Url on Notes page
- And this example

Decision Trees - RDD-based API

- Basic algorithm
 - Node impurity and information gain
 - Split candidates
 - Stopping rule
- Usage tips
 - Problem specification parameters
 - Stopping criteria
 - Tunable parameters
 - Caching and checkpointing
- Scaling
- Examples
 - Classification
 - Regression

[Decision trees](#) and their ensembles are popular methods for the machine learning tasks of classification and regression. Decision trees are widely used since they are easy to interpret, handle categorical features, extend to the multiclass classification setting, do not require feature scaling, and are able to capture non-linearities and feature interactions. Tree ensemble algorithms such as random forests and boosting are among the top performers for classification and regression tasks.

`spark.mllib` supports decision trees for binary and multiclass classification and for regression, using both continuous and categorical features. The implementation partitions data by rows, allowing distributed training with millions of instances.

Ensembles of trees (Random Forests and Gradient-Boosted Trees) are described in the [Ensembles guide](#).

Basic algorithm

The decision tree is a greedy algorithm that performs a recursive binary partitioning of the feature space. The tree predicts the same label for each bottommost (leaf) partition. Each partition is chosen greedily by selecting the *best split* from a set of possible splits, in order to maximize the information gain at a tree node. In other words, the split chosen at each tree node is chosen from the set $\operatorname{argmax}_s IG(D, s)$ where $IG(D, s)$ is the information gain when a split s is applied to a dataset D .

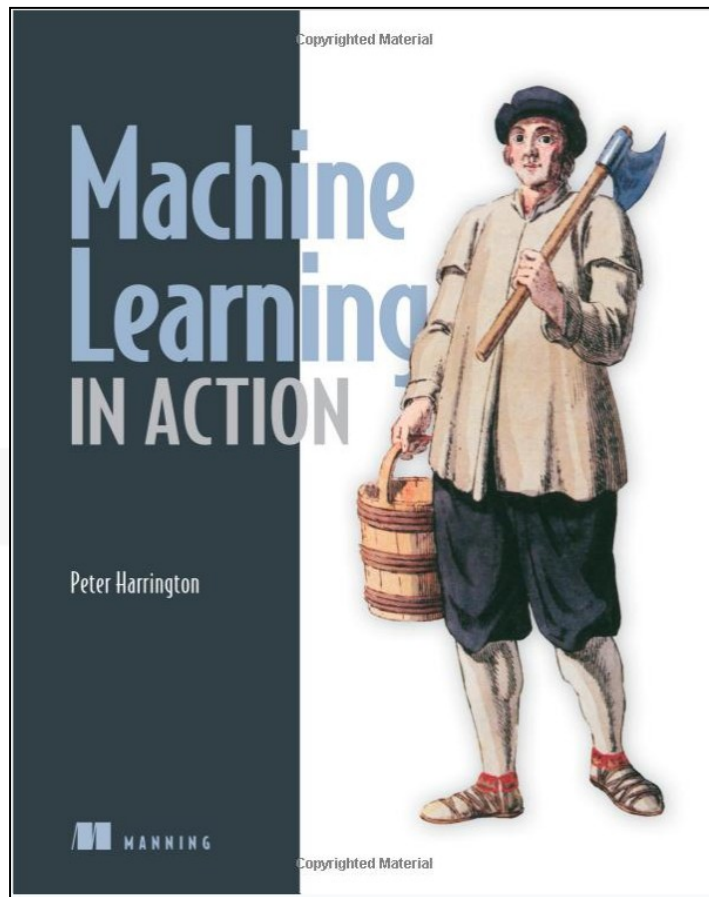


End of Unit:



Additional Detail:

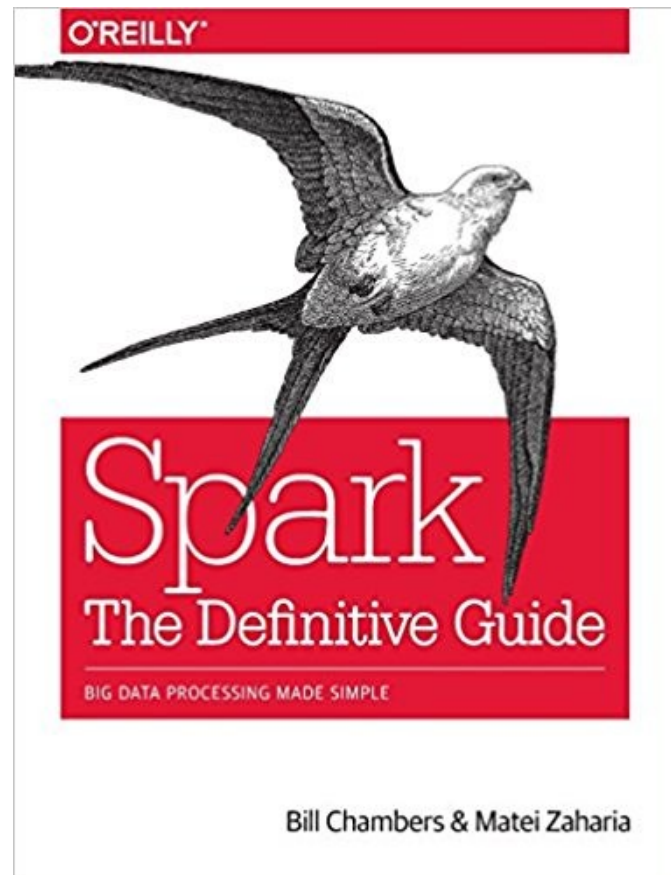
Machine Learning: A Must Read ?



- Older, Amazon
- Python
- Most examples also coded by hand; great for deep understanding
- Yes, Apriori
- No, Decision Tree Classifier

Machine Learning: A Must Read ?

- Amazon
- Python, Scala
- Newer
- *Complete*, if you already have some sense of the material
- No, Apriori
- No, Decision Tree Classifier
- Yes, others
- Yes, basics like pipelining



Apriori Algorithm: Source

Spark-ML overview-

Parallel compute engine, storage in DataStax

Based on simple math (algebra, some calculus, some set operands)

DataFrames (typed RDDs), and better parallelism to routines

Machine learning: *an algorithm that provides better results given more data*

Apriori algorithm-

Used for recommendation engines (customer who bought x also ..)

1994, Latin '*from before*' ..



Rakesh Agrawal,
MIT/Purdue



Ramakrishnan Srikant,
UW-Madison,
Google Fellow



Open until 10PM!
Silverthorne Lowe's

mylowes
Sign in



Departments

Search for it here...

Appliances > Washers & Dryers > Washing Machines > Front-Load Washers

LG TWINWash Compatible 4.5-cu ft High-Efficiency Stackable Front-Load Washer (Graphite Steel) ENERGY STAR

Item # 726520 Model # WM3670HVA

★★★★★ (128 Reviews)



In-use/lifestyle images - dryer and pedestals not included



\$749.00 Was \$999.00

SAVE 25% thru 09/13/2017

PLUS, up to \$190 in Rebates >

Manufacturer Color/Finish

Graphite Steel

Lowe's Extended Protection Plans [Learn More](#)

5 YEAR: \$139.97

3 YEAR: \$89.97

♥ SAVE

↗ SHARE

CHECK OTHER STORES

FREE Store Pickup

Unavailable for Pickup at
Silverthorne Lowe's

Delivery

Currently unavailable

Apriori

You put this in
your shopping
cart, then ..

Apriori

And after you
chose a washer
in stock,
16 straight
washers, no
other options

Customers Also Viewed



LG TWINWash Compatible 4.5-cu
ft High-Efficiency Stackable
Front-Load Washer (White)

★★★★☆ (82)

\$649.00 was \$799.00

SAVE 18% thru 09/13/2017



Maytag 4.3-cu ft High-Efficiency
Stackable Front-Load Washer
(White) ENERGY STAR

★★★★☆ (1208)

MSRP: \$799.00

Ends 09/13/2017



LG 4.5-cu ft High-Efficiency
Stackable Front-Load Washer
(Graphite Steel) ENERGY STAR

★★★★☆ (22)

\$799.00 was \$1,099.00

SAVE 27% thru 09/13/2017



Samsung 4.2-cu ft
High-Efficiency Stackable
Front-Load Washer (Platinum)

★★★★☆ (1133)

\$599.00 was \$899.00

SAVE 33% thru 09/13/2017



LG TWINWash
Compatible 4.5-cu ft High-Efficiency
Stackable Front-Load Washer (White)

★★★★☆ (2)

\$849.00 was \$1,099.00

SAVE 29% thru 09/13/2017